

LLM-Assisted IT Audit Risk & Control Assessment

An audit assistant that drafts control assessments from uploaded evidence — and checks its own citations.

This document explains what the program does, what it deliberately refuses to do, and how to operate it screen by screen. It is sent to you as a request for a practitioner's review: Part 2 sets out what I would most like you to test.

Synthetic data only. Every control, document, account name and configuration value shipped with this prototype is fabricated. It has no authentication, no access control and no encryption at rest. Please do not point it at real or client audit evidence.

Contents

Part 1 — What this program is

The problem it addresses, the REQUIRES / PROVES / INFERS / HUMAN-VERIFIES principle, and the guardrails

Part 2 — What I would like you to test

Six specific things to try, and what to send back

Part 3 — User guide: operating the console

Starting the application, the ten pages of the console, reading an assessment, and troubleshooting

Part 1 — What this program is

In one paragraph

This is a prototype **audit assistant**. An auditor uploads the evidence for a control — a policy document, a privileged-account export, a configuration listing — picks the control they want to test, and the system reads the evidence, retrieves the passages that bear on that control, and drafts an assessment: what the control *requires*, what the evidence *proves* (as verbatim quotes, each tied to a named file and an exact locator such as *page 3, section 4.2* or *rows 14, 27, 38*), what the model *inferred* beyond those words, and what a *human still has to check*. Every quote is then re-checked mechanically against the stored evidence before the auditor sees it, so a fabricated citation is caught by the system rather than by the reader. Nothing it produces is a conclusion: each assessment is held as *AI-generated* — *requires auditor review* until a named auditor records a decision, and that decision is stored as a separate record that never overwrites the model's.

The problem it is aimed at

Testing an IT general control is slow in a specific way. The evidence is long, the conclusion has to be defensible line by line, and the real risk in handing any of it to a language model is not that the model is slow — it is that the four kinds of statement in a working paper quietly blur into one another. A requirement gets written up as an observation. An inference gets presented as a quote. A gap in a spreadsheet export gets reported as a control failure.

The system is built so those four stay separate, structurally, all the way through:

	The question it answers
REQUIRES	What does the policy or control require?
PROVES	What does the evidence literally state? Verbatim, with a source and a locator.
INFERS	What travelled even one step beyond the literal words?
HUMAN-VERIFIES	What could not be established, and what must a reviewer check?

That is not a house style — it is the output schema the model must fill, the panel layout of every assessment screen, and the thing the citation validator checks.

The part I would most like a second opinion on

There is one distinction the whole design rests on: **INSUFFICIENT EVIDENCE** is not a mild deficiency. A deficiency says *the control did not operate as required*. Insufficient evidence says *nothing here settles the question either way*. If a privileged-account listing has no MFA column at all, the correct answer is not “MFA is missing” — it is “this listing cannot test MFA”. The system counts the two in separate tiles, colours them differently, and will downgrade a definite conclusion to **INSUFFICIENT EVIDENCE** when it rests on no verified citation. Whether that line is drawn in the right place, and drawn consistently, is the judgement I would most value yours on.

Guardrails, stated plainly

- It never asserts compliance with any law, regulation or standard. The prompt forbids it, and the framework references in the control library are labelled illustrative and unmapped.
- Every assessment is flagged for human review unconditionally. There is no setting that disables it.
- The AI record and the human record live in separate tables and are printed side by side in the report, with disagreements marked.
- Controls with no recorded auditor decision appear in the report as *PENDING AUDITOR REVIEW* and are excluded from every headline figure.
- Citation verification is **textual, not semantic**. **VERIFIED** means the quoted characters really are in the chunk cited. It does not mean the quote supports the conclusion drawn from it. That gap is a known limitation, not a bug.
- The risk score is a prototype additive model with my own weights. It is not an industry framework and is labelled as such wherever it appears.

Two things to know before you open it

All data is synthetic. Every control, document, account name and configuration value shipped with the prototype is fabricated. Please do not point it at real or client audit evidence: there is no authentication, no access control and no encryption at rest, and uploaded files are written to local disk in the clear.

By default there is no language model running. The prototype ships with a deterministic offline stand-in so it runs with no API key. A prominent **MOCK PROVIDER** badge in the sidebar tells you when that is what is answering. Results obtained with it measure the pipeline, not a model. Section 4 of the guide that follows explains how to point it at a real one.

Part 2 — What I would like you to test

Roughly an hour, if you can spare it. The fastest path is the one-minute tour on the first page of the guide: load the demo project, run the assessments in **mode C**, open one, record a review, generate the report.

Specifically

1. **Does an assessment read like a working paper?** Open any result and read the four panels. Would you be comfortable putting that in front of a client as a draft you were about to correct — or is it saying things an auditor would not say?
2. **Check a citation by hand.** Pick a quote, note the file and the locator, and open the source file yourself. Spreadsheet row numbers are the real 1-based Excel rows. Does row 14 mean row 14?
3. **Try to catch it out.** Upload a policy with no matching system export, or an export missing the attribute under test, and see whether it says `INSUFFICIENT_EVIDENCE` or invents a deficiency. This is the failure mode I care most about.
4. **Judge the risk ratings.** Expand the rating on a few findings — it explains its own score line by line. Are the bands where you would put them?
5. **Use the review screen as a reviewer.** Accept, reject, modify, request more evidence. Are those four actions enough? Is anything you would need to record missing? There is a **flag hallucination** checkbox — please use it whenever the model asserted something the evidence does not support, even if the validator did not catch it. That gap is itself a result.
6. **Read the generated report, especially Section 10 (Limitations).** Does it disclaim what it actually needs to disclaim?

What to send back

Anything at all, but these especially: where the language is wrong for an audit context; anything it stated that the evidence did not support; anything an auditor needs that is not on the screen; and your honest view on whether the effective / deficient / insufficient-evidence split is drawn correctly. The usefulness rating (1–5) on each review is a research measurement — please fill it in honestly rather than kindly.

Part 3 — User guide: operating the console

How to *use* the application, screen by screen, and — more importantly — how to read what it gives you back.

Companion documents: QUICKSTART.md (fastest path to a working demo) · SETUP.md (installation and configuration) · RESEARCH_NOTES.md (running experiments for a dissertation) · EVALUATION_METHODODOLOGY.md (what the metrics mean).

Before you start: what this tool is, and what it is not

This is an **audit assistant**. It reads the evidence you give it, compares it against a control you select, and drafts a finding with citations you can check. It does not decide anything. Every assessment it produces is marked *AI-generated* — *requires auditor review* and stays in that state until you record a decision.

It will never tell you an organisation is compliant with a law, regulation or standard. It is not capable of that judgement and is explicitly instructed not to attempt it.

All data shipped with the prototype is synthetic. Do not point this version at real confidential audit evidence: there is no authentication, no access control, no encryption at rest, and evidence files are written to your local disk in the clear.

Starting the application

```
cd "/Users/tingo/Documents/IT AUDIT TOOL" && .venv/bin/python run.py
```

Two services start:

Service	URL	What it is for
Audit console (Streamlit)	http://localhost:8501	Everything below
REST API (FastAPI)	http://127.0.0.1:8000/docs	Programmatic access, scripted experiments

The console works on its own — `.venv/bin/python run.py ui` is enough. It calls the service layer in-process, so nothing breaks if the API is not running.

No API key is required. The system ships with a deterministic offline provider.

If a port is already busy, the launcher stops both services and says so. Free the port and start again:

```
lsof -ti:8501 | xargs kill -9; lsof -ti:8000 | xargs kill -9
```

The one-minute tour

If the database is empty, the console opens on an empty Dashboard with a single button: **Load demo project + synthetic evidence**. Press it. It creates the *Privileged Access Management Audit* engagement, scopes five controls, and ingests nine synthetic evidence files (policies in `.docx` / `.pdf` / `.txt`, exports in `.csv` / `.xlsx`).

Then: **Assessments** → **select all in-scope controls** → **Run** → open any result → **Human Review** → record a decision → **Reports** → **Generate**.

That is the whole workflow. The rest of this guide explains each step properly.

The sidebar

Present on every page.

- **Audit project** — the engagement everything else is scoped to. Change it here and every page follows. Projects created by the evaluation harness never appear in this list; they are experiments, not engagements.
 - **Reviewing auditor** — your name. It is stamped on every review you record and on every report you generate, so the audit trail attributes work to a person.
 - **Provider badge** — shows which brain is answering. A prominent **MOCK PROVIDER** badge means the deterministic offline stand-in is running. Take that badge seriously: results obtained with it measure the pipeline, not a language model.
 - **Demo data** — loads the demonstration engagement. Safe to press twice; files already ingested are skipped.
-

Page 1 — Dashboard

The engagement at a glance.

The seven headline figures: controls in scope, controls assessed, effective, potential deficiencies, insufficient evidence, high-risk findings, reviews pending. Counted over the *latest assessment per control*, excluding anything produced by the evaluation harness.

Read this carefully: **INSUFFICIENT EVIDENCE** is not a mild deficiency. It is counted in its own tile and drawn in its own colour throughout the console, because it is a different kind of statement. A deficiency says *the control did not operate as required*. Insufficient evidence says *nothing here settles the question either way*. Reporting one as the other is a methodological error, and the console is built to make that hard.

Requires your attention — the queue, most severe first: high-risk findings and anything without an auditor decision. Nothing here is settled until you record one.

Evidence coverage — which in-scope controls the pipeline could actually cite evidence for. Note the distinction it draws: *assessed, nothing cited* is an evidence gap, not a control failure; *not assessed* is reported as unknown rather than as uncovered. Evidence files are not mapped to controls by hand — retrieval decides what bears on a control when the control is assessed.

Recent activity — the append-only trail, newest first.

Page 2 — Audit Projects

Create and manage engagements. Fields: name, audit area, description, audit period, auditor, status (**PLANNING** → **FIELDWORK** → **REVIEW** → **COMPLETED** → **ARCHIVED**).

Open a project for four tabs:

- **Controls in scope** — add controls from the library, with an optional scope note. A control must be in scope before it can be assessed.
 - **Assessment progress** — what has been run and what has not.
 - **Edit / Delete** — deletion is confirmed, and cascades to that project's evidence, assessments and reviews.
-

Page 3 — Controls

The control library: 14 synthetic IT controls, including the five the research brief specifies (**CONTROL-001** MFA for privileged accounts through **CONTROL-005** password policy). Browse by category or search.

Each control carries: objective, description, risk addressed, **expected evidence**, **assessment criteria**, inherent risk, privilege level, data sensitivity, and retrieval keywords.

Two fields do most of the work and are worth understanding if you add your own controls:

- **Assessment criteria** must be *testable* — a threshold a person could check. Compare “MFA should be used” (useless) with “100% of accounts in the privileged account listing have `MFA_Status = Enabled`” (checkable). The quality of these criteria largely determines the quality of the assessment.
- **Retrieval keywords** should include the literal column names you expect in the evidence (`MFA_Status` , `Last_Login`). Retrieval matches against real export headers.

Framework references are illustrative and synthetic. They are not an authoritative mapping to any published framework.

Page 4 — Evidence

Upload and inspect the material the assessment will read.

Uploading: drag in one or more files (`.pdf` , `.docx` , `.csv` , `.xlsx` , `.txt` , `.md` , `.json`), set an **evidence type** and a description. The type matters — it helps retrieval tell a policy from a system export.

On upload each file is hashed (SHA-256), stored, parsed into chunks, and embedded. You see the parse result immediately: page/row/chunk counts and any warnings. A file that cannot be parsed is recorded with its error rather than silently dropped.

Parsed chunks is the screen that makes citations checkable. Every chunk shows the exact text the model can see and the locator it will cite — `page 3, section '4.2'` for a document, `rows 14, 27, 38` for a spreadsheet.

Spreadsheet row numbers are the real 1-based rows as they appear in Excel, so a citation to row 14 means row 14 when you open the file.

Legacy `.xls` is supported as well as `.xlsx` ; audit exports still arrive in the old format.

Page 5 — Assessments

Where the AI does its work.

Running one: pick in-scope controls, pick an experiment mode, run. Expect a few hundred milliseconds per control with the mock provider; seconds with a real model.

The three modes are the research conditions, and for normal audit use you want **C**:

Mode	What it does	Use it for
A — raw evidence	No retrieval. Raw file text, truncated, minimal prompt, no citation scaffolding, no safety rails .	The research baseline only. Deliberately weak.
B — RAG + control	Retrieval, the full control requirement, structured output, citations validated.	Comparison condition.
C — RAG + workflow	B, plus an evidence-sufficiency pre-check, a self-critique pass, safety rails and risk scoring.	Normal use.

Mode A is *intentionally* unprotected — railing the baseline would erase the difference the experiment exists to measure. Never treat a mode A output as audit work.

Reading an assessment

The detail view is the heart of the system. It separates four different kinds of statement and never lets them blur:

Panel	The question it answers
Control requirement	What does the policy or control REQUIRE ?
Cited evidence	What does the evidence PROVE ? Verbatim quotes, each tied to a source.
Inferences	What did the AI INFER beyond the literal evidence?
Human verification required	What must YOU check before any of this can be relied on?

Also shown:

- **Conclusion, sufficiency and what the model says is missing** — the status, how completely the evidence covered the control, and which expected artefacts were absent.

- **Prototype risk rating** — score, band, and the contribution of each factor. Expand it; the rating explains itself line by line.
- **Cited evidence** — each citation as a card: filename, locator, the verbatim quote, and a **verification verdict**. Expand to see the full source chunk in context.
- **Retrieval transparency** — which chunks were retrieved, their scores, and which of them the model actually cited. A large gap between retrieved and cited is worth a look.
- **Citation validation** — verified / partial / fabricated counts, grounding rate, unsupported claims, and any safety rails that fired.

The verification verdicts

Every citation is mechanically re-checked against the stored evidence. The model is not trusted to cite honestly; it is checked.

Verdict	Meaning	What you should do
VERIFIED	The quote is genuinely in the chunk cited.	Spot-check as usual.
PARTIAL	Partially matched. Possibly paraphrased.	Read the source chunk.
UNVERIFIED	Real chunk cited, but no quote given.	Read the source chunk.
FABRICATED	The quote is not in that chunk, or the chunk does not exist.	Treat the whole assessment as unreliable.

The safety rails: if a definite conclusion (**EFFECTIVE** or **NOT_EFFECTIVE**) rests on no verified citation, the system downgrades it to **INSUFFICIENT_EVIDENCE** and records why. It will not let the model assert something it cannot evidence. The original conclusion is kept in the audit trail — nothing is silently rewritten.

Page 6 — Findings

The findings register for the engagement. Filter by status, risk, control and review state; sort by risk; expand any row for the finding, its risk rationale and its evidence references. **Export to CSV** for your working papers or your write-up.

The table shows the AI status and the final auditor status side by side. Where they differ, the row is marked.

Page 7 — Human Review

The review queue — the step that makes this an assistant rather than an oracle.

For each pending assessment you see the **AI proposal** beside **Your decision**:

Action	When to use it
Accept Finding	The conclusion and its evidence hold up.
Reject Finding	The conclusion is wrong.
Modify Finding	Broadly right, but the status, risk or wording needs to change.
Request More Evidence	Untestable as it stands; name what you need.

You can override the **final status**, **final risk rating**, **final finding** and **recommendation**, and add comments. Two fields exist for the research and are worth filling in honestly:

- **Usefulness rating (1–5)** — how much the AI draft actually helped.
- **Flag hallucination** — the AI asserted something the evidence does not support. Use this even when the validator did not catch it; that gap is a finding in itself.

Review time is recorded, counted only while the item is actually open. It is a research measurement — do not leave an item open on another tab and expect the number to mean anything.

The AI assessment and your decision are stored as **separate records**. Your decision never overwrites the AI's; keeping both is what makes human–AI agreement measurable.

Page 8 — Evaluation

The research dashboard. Not needed for ordinary audit work.

Six synthetic datasets with known correct answers. Run experiments A/B/C over any subset, then read: the mode comparison, per-class precision/recall/F1, the confusion matrix, deficiency-detection precision/recall/FPR/FNR, citation and grounding rates, latency, and human–AI agreement with Cohen's kappa. Export to CSV.

Two things to look at before quoting any number:

- **n is displayed beside every metric**. With six datasets, $n=6$. That is far too small for a confident claim, and the **caveats** list says so automatically. Report n every time.
- **Grounded accuracy** counts a prediction as correct only if it *also* carries a verified citation. The gap between accuracy and grounded accuracy is the share of right answers nobody can check. In the shipped configuration mode A scores 0.667 accuracy and **0.000** grounded accuracy — every correct answer it produced was unverifiable.

Page 9 — Reports

Generate the audit report for the current engagement, in Markdown or HTML. Preview it in-page, download it, and see previously generated reports.

Ten sections: Executive Summary, Audit Scope, Controls Tested, Evidence Reviewed, AI Findings, Risk Ratings, Evidence References, Human Auditor Decisions, Recommendations, Limitations.

Throughout, **AI-generated assessment** and **Final auditor assessment** are labelled separately and never merged. Controls you have not reviewed yet appear as *PENDING AUDITOR REVIEW* and are excluded from the headline conclusions. Section 4 records each evidence file with its SHA-256 hash. Section 10 states plainly what the report cannot establish — read it before sending the report to anyone.

Page 10 — Settings

Read-only configuration. API keys are masked; the console never displays a secret.

Shows the active provider and model, embedding provider, retrieval strategy and top-k, chunk settings, database URL, prompt version and app version — the values you need to record for a reproducible experiment.

Data management: seed the demo project, generate synthetic datasets, and a reset action behind a typed confirmation.

Configuration is changed by editing `.env` and restarting, not from this page — an unauthenticated endpoint that could replot the model on a tool holding audit evidence would be a security hole.

Using a real language model

The mock provider is a deterministic rule engine, not a model. To run against a real one, edit `.env`:

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o-mini
LLM_API_KEY=your-key-here
```

Any OpenAI-compatible endpoint works — add `LLM_BASE_URL=http://localhost:11434/v1` for Ollama, or your vLLM / LM Studio address. Restart, and check the Settings page or the provider badge to confirm what is actually answering.

Never commit `.env`. It is already in `.gitignore`.

A suggested working order

1. **Audit Projects** — create the engagement, set the period and auditor.
2. **Controls** — review the criteria for the controls you intend to test.
3. **Audit Projects** → **Controls in scope** — scope them.
4. **Evidence** — upload policies *and* system exports. Both: the whole point is comparing what a policy requires against what an export shows. A spreadsheet alone cannot test it.
5. **Assessments** — run mode C over the scoped controls.
6. **Assessments detail** — read each one. Check the citations against the source chunks.
7. **Human Review** — record a decision on every assessment.

8. **Findings** — export the register.

9. **Reports** — generate, read Section 10, then circulate.

Troubleshooting

Symptom	Cause and fix
Port 8501 is already in use , both services stop	A previous run is alive. <code>lsof -ti:8501 \ xargs kill -9</code> , then restart.
Everything returns <code>INSUFFICIENT_EVIDENCE</code>	Usually correct — the evidence genuinely lacks the attribute. Check the <i>missing evidence</i> list, then confirm on the Evidence page that the file parsed and the relevant column exists.
An assessment cites nothing	Retrieval found nothing relevant. Check the control's retrieval keywords match the column names in your export.
A <code>.xls</code> upload fails	Confirm <code>xlrd</code> is installed (<code>.venv/bin/python -m pip install xlrd</code>). If the file is actually an <code>.xlsx</code> renamed to <code>.xls</code> , re-save it properly.
Project list is cluttered	Evaluation projects are hidden automatically. If you see them, you are looking at an API call passing <code>include_evaluation=true</code> .
Results changed between runs	With the mock provider they should not. Check <code>MOCK_HALLUCINATION_RATE</code> is <code>0.0</code> and that the prompt version in Settings is unchanged.
Want to start clean	<code>.venv/bin/python run.py reset</code> (destructive, asks for confirmation).